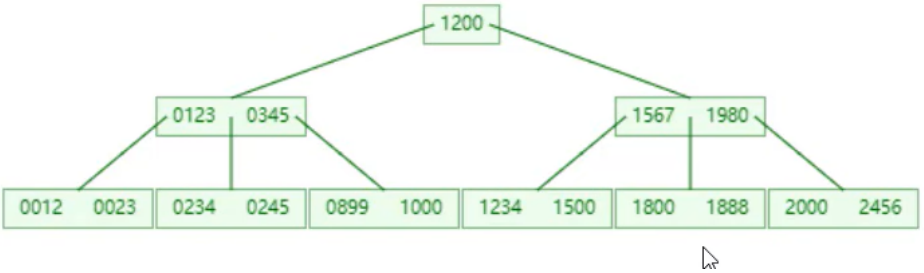
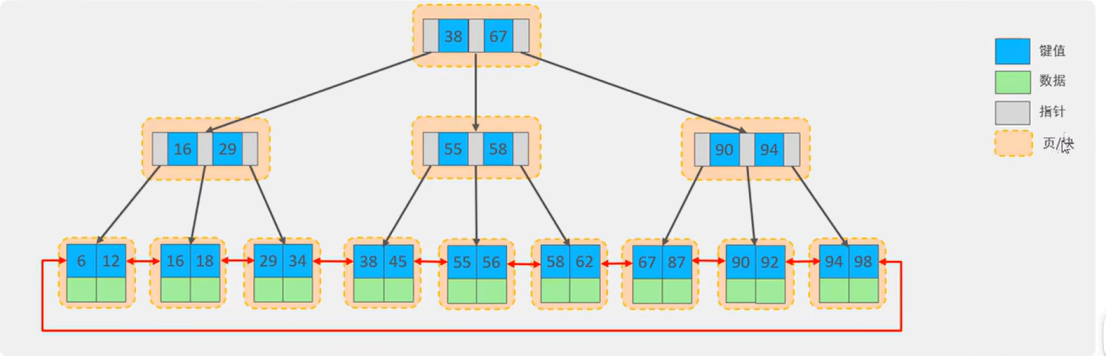
sql

索引

1. 索引的优缺点：提高查询的效率，提高数据排序的效率（树的特性）；占用存储成本，增删改的时候需要插入索引，性能降低。
2. 传统二叉树的缺点：顺序插入时会形成一个链表（红黑树解决）；大数据量的情况下，层级较多。
3. B树(key/指针)：中间元素向上分裂，示例：



4. B+树：所有数据存放在叶子结点，同时在叶子结点加上双向链表（mysql的优化），示例：



5. 一级索引和二级索引：一级索引中存放的是key+行数据；二级索引存放的数据是key+一级索引的key。回表查询也就是指where中使用到二级索引时，先通过二级索引找到一级索引的Key，再从一级索引中得到具体的数据。
6. 索引使用的原则：
    - 联合索引遵循最左前缀法则：where中设计的字段满足联合索引从左到右的顺序，如果中间跳过字段，则后续的字段不走索引（与where的顺序无关）。
    - 索引不能使用运算符，**否则失效。**
    - 字符索引要加上单引号，**否则失效。**
    - 模型查询如果只能后缀模糊，**否则失效。**
7. 使用字符串索引时字符串过长时可以使用字符串前缀作为索引。

Java面试基础

基础篇

java的四个特点：面向对象编程，跨平台能力强，多线程，强类型语言。

1. 三大特征：封装，继承，多态（静态多态和动态多态）
2. 面向对象和面向过程的区别：面向过程使用函数来完成，面向对象通过各种类来完成不同的功能实现。
3. 类和对象的基本概念：类是对象的模板，对象是类的实例。

4. java中的8种基本数据类型。
5. 重载和重写的区别：重载是子类中同一个名称的方法根据不同参数实现不同的功能；重写是针对父类不同子类的相同方法。
6. hashCode知识点：hashCode相同，equal不一定相同，因为可能会发生冲突；hashcode不相同，equal一定不相同。通过这种方式减少HashSet的equal对比次数。
7. ==和equal的区别：==针对基础类型的时候比较值，针对引用类型的时候比较引用地址；equal比较对象的引用地址，需要在类中重写equal方法。**重写equal时首先使用==来判断两个对象的引用类型是否相同，在进行字段级别的比较。**
8. String，StringBuilder和StringBuffer之间的区别是什么？

   String不可变，StringBuilder和StringBuffer可变；String和StringBuffer（使用了线程锁）是线程安全的，StringBuilder是非线程安全的。

   使用场景：对字符串少量改动时使用String，大量改动是使用StringBuilder，使用到多线程的时候使用StringBuffer。
9. String为什么是final关键字修饰：安全，由于不会被继承，所以就不会被修改；string使用频率较高，直接使用final对类进行修饰省事。
10. super和this的异同：super调用父类的方法和变量名，this使用自身的构造方法和变量名；super和this必须放在构造方法的第一行 ，并且不能同时出现。

集合篇

1. ArrayList和LinkedList的联系和区别：两个都是线程不安全的；ArrayList使用Object数组存放数据，LinkedList使用双向链表存放数据，这就导致了数据的存放和访问的效率不一样。
2. HashMap的底层特点：在JDK7之前都是使用数组+链表的形式来对数据进行存储，这样如果发生哈希冲突的话会使得链表越来越长，查询效率变低；JDK8之后使用数组+链表+红黑树的形式来存储，当链表数据个数超过8个就使用红黑树来存储数据，提高查询效率。
3. HashMap为什么是线程不安全的：当有多个线程插入的元素出现碰撞的时候可能会出现覆盖数据问题。
4. HashMap如何实现线程安全？使用ConcurrentHashMap实现了线程安全。
5. HashMap底层实现

多线程

1. 并发和并行的区别
2. 进程和线程：进程是资源分配的单位；线程是调度和执行的最小单位。
3. 线程同步的方法：同步方法；同步代码块。
4. 线程池：核心线程池，最大线程池，队列和拒绝策略。

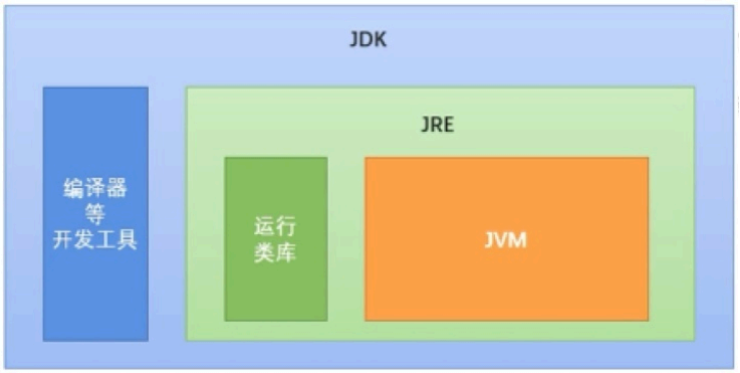
   判断核心线程池是否已满，没满则创建一个新的工作线程来执行任务。

   判断任务队列是否已满，没满则将新提交的任务添加在工作队列。

   判断整个线程池是否已满，没满则创建一个新的工作线程来执行任务，已满则执行饱和（拒绝）策略。
5. synchronized关键字；懒汉式和饿汉式的线程安全问题？
6. volatile关键字？
7. 线程死锁的原因（4），解决死锁的方式（3）。

JVM

1. JDK，JRE和JVM的区别。



数据库专题

1. SQL解析顺序。

```
(9)  SELECT
        (10) DISTINCT column,
        (6)  AGG FUNC(column or expression), ...
(1)  FROM left tab1
        (3)  JOIN right tab2
        (2)  ON tab1.column = tab2.column
(4)  WHERE constraint_expression
(5)  GROUP BY column
(7)  WITH CUBE ROLLUP
(8)  HAVING constraint_expression
(11) ORDER BY column ASC|DESC
(12) LIMIT count OFFSET count;
```

2. MYSQL中的日志文件：回滚日志，重做日志和二进制日志。

   回滚操作用于撤销没有写入到二进制日志中的事务。

   重做日志用于完成已经记录到二进制日志中的事务。

3. 三大范式的区别

4. 索引的作用是什么?

Hadoop

1. Hadoop是什么，优缺点。

   Hadoop是一个分布式系统架构，由HDFS，MapReduce，yarn组成。

   优点：

   1. 高可靠性：多副本机制。
   2. 高扩展性：通过拓展廉价的机器提高整体集群的性能。
   3. 高效性：MR框架将一个大型的任务分成多个小任务进行执行。
   4. 高容错性：任务失败之后自动在其他节点重启任务。

   缺点：

   1. 不适用低延迟访问。
   2. 不能高效地处理小文件。

3. 不支持多用户写。

2. HDFS的定义。

   HDFS是一个高可靠，高拓展的分布式文件系统。采用主从和主备的设计模式。由以下三部分组成：

   NameNode：存储元数据信息。

   SecondaryNamenode：用于帮助Namenode分担压力，合并Fsimage和Editlog。

   DataNode：存储实际的数据。

   拓展：

   主从和主备的区别：主从是指主节点（NameNode）和若干个从节点（Datanode）；主备是在主从的基础上加上了备用主节点（standby namenode），当主节点发生故障，会从备用主节点中选举出新的主节点。

3. Hadoop的读写文件流程：

   读文件：

   1. 客户端向Namenode发起读文件的申请，用户获取数据所在的位置。
   2. 客户端根据数据所在的位置就近向DataNode读取数据。
   3. DataNode以Packet为单位向客户端传输数据。

   写文件：

   1. 客户端向NameNode发起写文件的申请，Namenode返回父目录是否存在等相关信息。
   2. 如果存在则返回写入文件的DataNode地址。
   3. 客户端在各个DataNode之间建立一条通信管道，以Packet为单位来传输文件。

4. MR的任务执行流程。

   Split阶段：切分文件，其中切分之后的每一个文件对应一个Task。

   Map阶段：将文件的数据切分成KV键值对的形式。

   Shuffle阶段：

   - Map端shuffle：KV键值对被放入环形缓冲区，当达到80%的内存空间的时候对里面的数据进行快排并且溢出到磁盘中生成小文件。当生成足够多的小文件的时候进行归并排序生成一个有序的大文件。
   - Reduce端shuffle：Ruduce根据自己的分区向DataNode去获取自己对应的文件，然后进行归并排序将生成一个有序的大文件。

   Reduce阶段：各个Reduce根据得到的有序大文件执行运算得到最终结果。

5. yarn是什么，三种资源调度器的区别是什么?

   yarn是一个资源管理器，由四部分组成：ResourceManager，ApplicationMaster，NodeManager，Container。执行任务流程如下：

   1. 用户将应用程序发送到RM
   2. RM为AM申请资源，向NodeManager提出启动Container。
   3. AM与启动container的NM通信启动Task。
   4. 所有任务结束之后，AM向RM进行注销。

   三种常见的调度器：

   先进先出调度器：根据任务来的顺序分配资源，会导致紧急的任务无法得到及时的处理。

容量调度器：设置每个队列的最小容量，来保证小任务能够得到充分的资源。比如为每一个部门来设置容量大小。

公平调度器：提前为每一个队列来设定权重，保证重要的任务能够得到足够的资源。

6. Hadoop读写流程三种容错措施
    1. 写数据是部分datanode出现异常：移除当前异常的datanode，在剩下的datanode里面进行传输，同时namenode重新分配一个datanode保证副本数量。
    2. 读数据时发生中断或者数据有问题：读取其他存放数据的datanode。
7. HDFS副本机制以及存放策略

    在HDFS中默认会存放3个副本，防止机器发生损坏的时候依旧能够从其他机器读取数据，为了提高读取的速度，有些公司会将副本数量增加；存放策略是第一个副本位于客户端的节点上，第二个位于和第一个节点相邻的机架上，第三个位于和第二个的机架上。
8. HDFS块大小的设置

    HDFS块公司一般默认是256MB，如果文件太大会将文件进行拆分成若干个block。

    扩展：

    为什么块大小是128MB?

    块太小会导致寻址时间过长，块太大会导致数据IO时间过长。
9. HDFS常用命令

    HDFS dfs + linux常用命令（如-ls， -cat，-touchz）
10. MR中split阶段InputFormat类

InputFormat分为TextInputFormat和ConbineTextInputFormat。其中前者不会自动合并小文件，后者会自动合并小文件。

拓展：

为什么块大小等于切片大小：防止跨越块导致需要从两个节点取数据导致网络传输。

11. 为什么MR的执行效率很低?
    1. 频繁的落盘操作：无论是中间的排序还是文件合并操作，都需要进行落盘；而且不支持连续的shuffle，每次MR任务结束之后也需要进行落盘操作。
    2. MR都是java的进程任务，启动和销毁都需要时间。

Hive

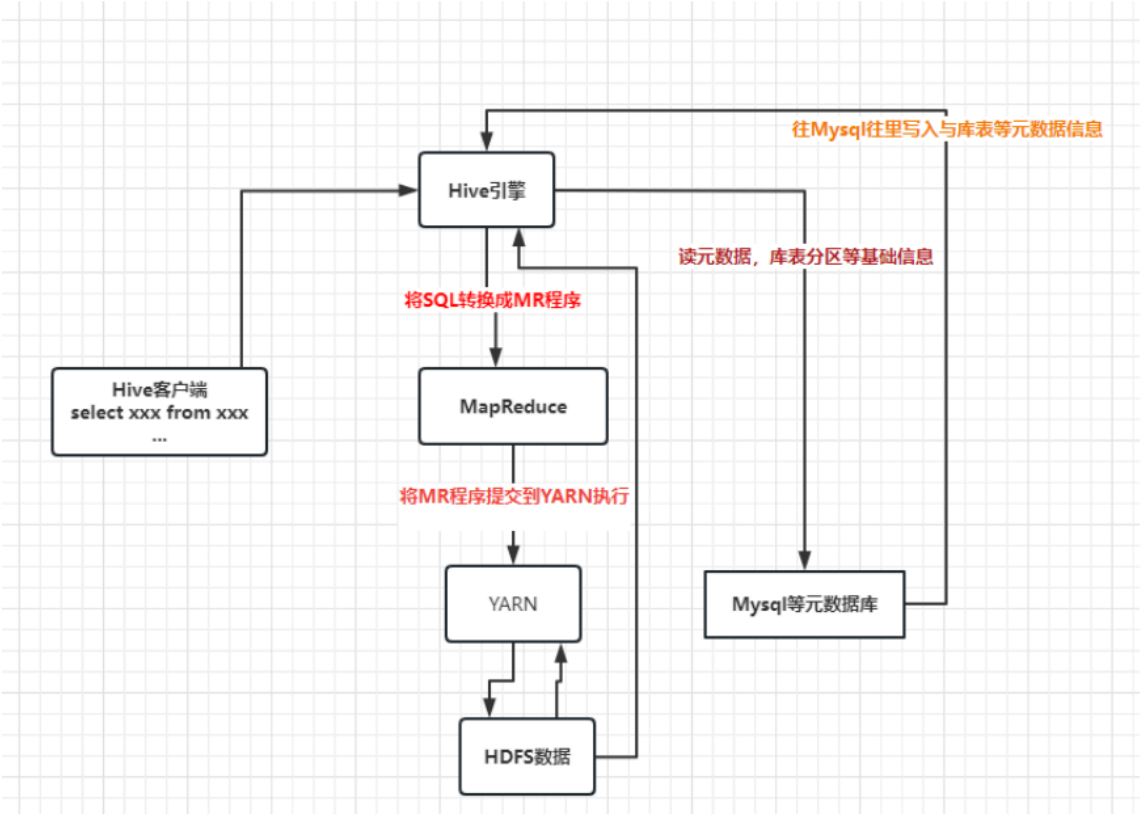
1. hive是什么，主要架构是什么?

    hive的本质就是将HQL转换成MR任务。由客户端和服务端两部分组成：

    客户端：CLI，GUI。

    服务端：Driver（Compiler，Optimiter，Executor），metastore，Thrift Server（其他语言的接口）
2. hive的任务执行流程

    Hive和Hadoop的交互关系如下所示：得到客户端之后sql之后Hive引擎结合元数据信息来生成MR任务，交到Yarn进行执行，最后得到结果。



3. Hive的任务转换执行流程

   先调用Driver生成语法树。

   检查语法树是否存在问题。

   根据语法树来生成逻辑执行计划。

   通过元数据来优化逻辑执行计划。

   将逻辑执行计划转化成物理执行计划，生成MR任务。

4. spark比MR快的原因是什么?

   - Spark的落盘次数比MR少：spark只在发生宽依赖的时候才会执行shuffle操作，而每一个MR任务都会执行shuffle。
   - Spark的Bypass操作：只在特殊场景下进行排序操作。
   - Spark的Persist：可以对常用的数据进行持久化操作。
   - Spark是线程级别的任务：线程级别的任务可以减少任务拉起所消耗的资源，可以共享进程中的资源。

   拓展：

   bypass操作什么情况下会触发：

   非聚合类操作的时候并且分区数小于指定阈值的时候。这个时候会将数据直接hash shuffle打散成若干个小文件，然后reduce端直接根据hash索引来获取数据。

5. hive中的表类型有哪些?

	分区表	**分桶表**
数据存储形式	每个分区以文件夹的形式存在，可以随意新增	每个分桶以文件的形式存储在同一个目录下，固定桶个数，根据cluster by的key进行hash存储。

	分区表	分桶表
优点	主要提高查询效率，减少扫描的数据量	在执行join查询的时候可以提高效率，比如bucket-mapjoin
关键字	partitioned by	clustered by .key sorted by .key into X个buckets

6. 静态分区和动态分区的区别?

   静态分区：在插入数据的时候需要手动指定插入的分区，稳定不容易出错。在企业中一般都用静态分区。

   动态分区：在插入数据的时候根据数据自动推测分区字段，容易产生大量小文件且不可预测。

7. hive中的表存储格式有哪些?

   行式存储：常见的有textfile，suquenceFile。其中suquenceFile的压缩比textfile高。

   列式存储：常见的有ORC和Parquet这两种。

	行式存储	列式存储
存储方式	每一行的所有列放在一起存储	每一个列的所有数据放在一起存储
特点	插入和更新操作快，只需对一行数据进行操作	能够更有效的进行压缩；同时访问固定列的时候更加快捷
适用场景	OLTP（在线事务处理）	OLAP（在线分析处理）

   拓展：

   ORC和Parquet的区别是什么：ORC是Hive社区开发的，**专用hive优化，对hive的数据格式友好**；parquet是twitter公司开发的，主要是**跨平台通用性。在hive中一般使用ORC，parquet格式spark引擎会用的多一些。**

8. hive中常见的压缩格式有哪些?

   常见的有Gzip（不可拆分），**snappy（不可拆分），LZO，ZSTD**这几种格式。企业中常用的snappy，LZO，ZSTD。

   企业中常见的组合就是ORC+snappy，因为ORC分为多个行组，这个时候在行组内部使用snappy进行压缩。然后在拆分的时候对行组进行拆分即可。

9. Hive中内部表和外部表的区别是什么?

   存储位置：内部表建表之后默认的存储位置位于warehouse中；外部表需要指定数据文件位置。

   生命周期：内部表我们删除之后对应的数据文件也会被删除；外部表不会。

   使用场景：在企业中我们大部分使用内部表，方便管理。

10. 增量表和全量表的区别是什么?

    增量表只保留当天新增的数据；全量表会保存截止到当前日期的全量数据。

    在授信域中，用户申请的流水表就是使用增量表来记录每天新增的数据；客户信息就使用全量表来保存最新客户信息即可。

11. 所有的Hive任务都会启动MR吗?

    不会，如果是简单的select可以直接通过Fetch task获取数据（直接从HDFS拿数据即可）即可。

11. 说一下哪些操作会导致发生shuffle?

    join操作，group by操作，count(distinct)，order by，开窗函数中的partition py。

12. 说一下Hive on spark和Spark on hive的区别是什么?

    Hive on spark是hive负责优化，而spark on hive是spark负责优化。hive都负责底层的存储，spark负责执行。

13. hive中cascade关键词的作用。

    在修改表的时候将旧分区中元信息也进行修改。一般我们都会加上这个关键词。

14. HQL和SQL的执行流程有什么区别?

    SQL会先进行join操作，但是HQL会先执行where操作减少数据量。

    整体的流程是:

    FROM-WHERE-JOIN-GROUP BY-HAVING-SELECT-ORDER BY-LIMIT

15. row_number、rank和dense_rank的区别

    row_nuber:12345

    rank:12334

    dense_rank:12334

16. Hive UDF的开发

    新建一个java类继承UDF，重载evaluate方法。

    打包成JAR上传到jiqun。

    在hive中进行导入。

17. 拉链表的使用场景和方法

    使用场景:

    1. 表中的部分字段会发生变化并且变化的比例很低，比如从申请状态变为通过状态。
    2. 需要获取用户的历史状态数据。

    特点：其中有start_time和end_time两个字段，最新的字段end_time = “9999-01-01”。

    拓展:

    拉链表的优缺点

    1. 优点:
        - 节约存储成本
        - 支持历史查询
    2. 缺点:
        - 历史数据修复成本高。
        - 维护成本高。

18. 缓慢变化维怎么处理?

    1. 如果业务不需要用到历史数据的话就直接进行覆盖。
    2. 需要保留历史数据:
        - 当变化不频繁的时候，我们可以选择直接增加维度行的方式（拉链表）。
        - 如果变化比较频繁：定期进行快照表。

19. 大表关联大表的情况怎么处理（比如100亿数据量关联10亿的数据量）?

1. 使用分区表或者分桶表来减少扫描的数据量，另外分桶表还是可以实现快速的join（分桶字段等于join字段）
2. 数据倾斜处理：进行加盐处理（会导致数据膨胀）
3. 使用map join，将小表加入到内存中减少shuffle的数据量。

20. 怎么提高一个表的压缩率?

    修改压缩格式。

    可以使用distribute by将相同字段放到同一个task（文件）中来提高文件压缩比。

21. 怎么将一个表均匀划分成N个文件大小?

    在insert的过程中使用distribute by floor(rand() * 10)来进行打散并且限制reduce的个数。

Hive优化

hive优化分为SQL优化和参数优化两部分。

1. SQL优化

过滤模式：

- 分区表：如果where中的字段属于分区字段，那么会在map前进行过滤，减少扫描量。
- 谓词下推：如果是外连接，主表中的字段应该放在where中，从表中的字段应该放在on中；如果是全连接，谓词下推失效，使用两个子查询进行连接；如果是内连接，默认在Map端进行过滤。
- 列裁剪：在select中过滤需要的字段，不会减少扫描量，减少之后任务的扫描量。

聚合模式：

conut(distinct 字段)：这种情况下之后产生一个reduce，效率低；使用group by+conut的方式将其划分为多个reduce进行处理。

order by：order by只会产生一个reduce任务。使用distribute by+sort by来产生多个reduce任务进行排序。

join模式：

map join：小表关联大表。

Bucket-mapjoin：同为分桶表，并且分桶数量为倍数关系。

SMB-mapjoin：在Bucket-mapjoin的基础上，sort by的字段也是分桶字段。

skew-join（只支持inner join）：自动根据热点键来进行负载均衡，划分到多个reduce中进行处理。不过和

2. 参数调优：

map和reduce的内存和核数调优：一般是1核2g。

map个数的控制：min.size和输入文件的个数以及文件是否可以拆分。

reduce个数的控制：可以直接指定reduce的个数；每个reduce处理的数据大小；reduce最大数量。

group by优化：map端聚合。

slow-start：在map端任务执行到指定进度时开始reduce的任务。可能会发生抢占资源，反复kill任务的情况。

3. 数据倾斜问题

   数据倾斜问题分为map端和reduce端的数据倾斜。

map端的数据倾斜是因为输入的文件大小不均匀且文件不可拆分。解决方式是map端小文件合并和修改文件的压缩格式。

reduce端的数据倾斜是因为热点键问题，在这里可以分为group by和join两种类型，但是无论是那种类型之前都要先回答首先通过过滤或者聚合等方式来减少数据量：

- group by类型：对热点值单独提取出来加盐进行二次聚合。
- join类型：如果热点值没有业务意义，比如是null值，可以在join之前直接将其过滤；如果有意义需要保留，也可以根据维表的大小来分为两种情况：
  - 维表不大：那就先对主表进行加盐处理，然后再对维表进行对应的数据膨胀，最后再进行聚合。
  - 维表较大：单独将热点值提取出来，然后对主表和维表进行加盐和数据膨胀处理，最后再union all。

4. 小文件合并问题

   关键参数：

   - hive.merger.smallfiles.avgsize：输出的文件平均大小。
   - mapred.min.split.size.per.rack：合并文件的最小大小。

   MR任务之后会判断输出文件平均大小是不是小于avgsize，如果小于的话另起一个map-only任务，根据min.size来合并小文件。

   存量小文件：重新启动一个任务，直接使用insert overwrite来合并小文件。

5. 小文件如何产生以及导致的问题。

   产生原因：

   - 流式数据的产生
   - 动态分区
   - spark分区数过多

   问题：

   - 元数据存储压力过大。
   - 影响MR任务的性能，因为map task对应一个小文件。
   - hdfs读写过程中小文件过多会导致寻址时间过长。

Spark

1. hive和spark怎么控制生成文件的个数。

   hive生成文件的个数等于reduce的个数，可以通过设置reduce的个数；每个reduce处理的数据量大小来控制。

   spark输出的文件个数等于最后的分区数，控制小文件可以使用repartition来对分区进行限制。

2. Spark的AQE。

   AQE的全称是自适应查询，根据前一个stage得到的数据信息来优化之后的stage执行计划。并不是所有情况下都会触发，只有当一个任务产生Shuffle的时候才会触发。

   作用：

   - 动态调整不同stage的partition个数。
   - 动态优化数据倾斜：自动检测热点键执行skew join。
   - 动态join类型转换：广播表和SortMergeJoin之间进行转换。

3. Spark的join类型有哪些?

Broadcast Hash Join：小表关联大表。

Shuffle Hash Join：两张表先根据join key进行分区，然后再将分区小的表作为Hash table进行连接。

Sort Merge Join：Spark默认的join方式，先根据join key进行分区，再对分区内的数据进行排序，再将相同分区的数据进行连接合并在一起。

笛卡尔积连接：当join key为空的时候触发。

广播嵌套循环连接：当小表关联大表时且非等值连接时。使用小表循环去每个Executor中执行join连接。

4. Repartition和coalesce的区别是什么?

   Repartition底层调用的是coalesce（shuffle=True）；coalesce只在分区变小的情况下生效，repartition分区变大变小都生效。

5. spark是什么，集群架构是什么?

   spark是一个基于内存的快速，通用的大数据计算引擎。具有MR的优点，但是落盘次数更少。由Master，Driver，worker，executor四部分组成。

6. spark执行任务流程

   1. 创建Driver，然后Driver向yarn的RM申请资源启动AM。
   2. AM向RM申请资源在nodemager上启动container，用于后续的executor
   3. AM和启动了container的NM发起请求启动executor。
   4. Driver的SparkContext分配Task给各个executor来执行，同时监控executor的状态。
   5. 应用程序结束，AM向RM注销。

   yarn-client和yarn-cluster的区别就在于Driver的位置不一样。client的位置位于客户端，cluster的位置位于集群内部的节点。

7. RDD的核心特性

   RDD的全称是弹性分布式数据集，本质就是存储对应操作的元数据。其中包含transformer和action算子

   弹性：容错机制，可以通过血缘记录来重新计算。

   分布式：数据存储分布在不同节点上。

   延时计算：只有遇到action算子的时候才会触发执行。

   拓展：

   如何理解RDD的弹性?

   1. 自动进行内存和磁盘数据存储的切换（会进行数据溢出）
   2. 基于血缘的高效容错。
   3. task失败会自动进行重试。
   4. checkpoint和persist操作（持久化和检查点）。

8. RDD的五大属性：本质相当于是元数据信息存储。
   - 分区列表：数据分片信息，决定并行度。
   - 分区器：当前RDD如何进行分区（hash分区器和range分区器）
   - 计算函数
   - 依赖关系：记录父RDD的依赖（宽窄依赖）

- 首选位置：选择最佳计算位置，移动计算而不是移动数据。比如窄依赖就可以在父RDD上进行运算。

9. 常见的RDD算子有哪些?

   根据返回值是否是RDD来判断类型，action算子返回RDD类型数据。

   Transformer算子：filter，map，union等。

   Action算子：count，collect，reduce等。

10. cache和checkpoint的区别

    1. cache将数据存储在内存中，底层调用persist，不会截取血缘关系。
    2. checkpoint将数据存储在HDFS中，会截取血缘关系，需要启动一个额外的任务。
    3. 可以在checkpoint之前使用cache避免产生一个新的任务，这样就直接将内存中的数据拷贝到HDFS即可。

    拓展：

    1. 对checkpoint的理解?

       在以下两种情况下我们会对RDD加检查点：

       1. DAG过长，如果重算开销过大。
       2. 在宽依赖上做checkpoint的收益更大（因为对应多个子RDD，出现问题的可能性更大）

       checkpoint相当于是对数据的备份，血缘相当于是对记录数据的备份，结合起来就可以实现在出问题时不用从头进行计算。

    2. 怎么手动实现checkpoint?

       ```
       spark.sparkContext.setCheckpointDir("hdfs://path/to/checkpoint") # 设置保存的路径
       rdd.checkpoint()  # 手动实现RDD保存
       ```

11. spark和MR的区别

    计算模式：spark以内存计算为核心，减少了磁盘I/O次数；MR每个任务都需要进行落盘操作。

    执行引擎：spark有专门的DAG任务调度引擎，用于优化任务执行顺序，灵活调度资源。

    编程模式与语言支持：spark提供RDD编程模型，支持多种语言；MR以键值对编程为主，远程支持的语言是java。

    适用场景：spark适用于机器学习，实时数据处理；hadoop适合处理大规模的离线任务。

12. Spark的shuffle流程

    shuffle write：shuffle操作意味着发生了宽依赖，需要将数据进行重新分布，spark会将不同的数据放到不同的executor中进行聚合，其中每个task会生成一个数据文件和对应的索引文件。

    shuffle read：通过索引文件，主动拉取属于自己的数据。

    spark和Hive的shuffle流程类似，但是spark中每个map任务只会产生一个数据文件，并且按照分区索引，减少了IO次数；同时支持堆外内存和bypass操作，避免不必要的排序；不用重跑所有数据；连续的shuffle在相同节点进行。

    拓展：

    shuffle的相关调优：

    1. 调整map端缓冲区的大小
    2. 调整reduce端缓冲区的大小（拉取数据）
    3. 调整reduce拉取重试次数。

4. 调整reduce拉取数据等待间隔。

13. spark中driver和executor是怎么交互的?

    通信方式：首先driver和executor是通过netty来进行通信的，保证了消息的及时性。

    任务分配：driver会根据整个集群资源的情况来考虑task的分配。

    数据分配：如果在多个executor上都会用到同一个数据集，会通过广播的方式来发送到各个executor上；如果不是就直接从数据所在节点上取数据。

    容错处理：executor要定时向driver汇报当前的处理情况，以及心跳机制判断当前executor是否还存活。

    拓展：

    广播机制的流程：

    1. 初始化的时候在Driver端会有一份数据。
    2. task如果想要使用广播变量，先从executor中的blockmanager中去尝试获取数据，如果没有就从driver端去拉取数据，此后的executor的task都可以直接使用blockmanager中的广播变量。当然除了从driver端拉取广播变量，也可以从其他节点收到blockmanager中去拉取变量。

    广播变量的特点：

    1. 只读变量，executor只能读取变量，不能修改变量。
    2. 多个task共享executor中的广播变量。

14. hive的底层引擎有哪些?

    一般我们使用的就是mapreduce，当然也可以使用spark来作为hive的执行引擎，即hive on spark。这个时候通过spark来执行具体的任务，好处就是IO操作；提高效率。

15. spark什么时候用到内存，什么时候用到磁盘?

    内存：连续的shuffle操作；广播变量；RDD缓存

    磁盘：shuffle溢写；RDD的持久化操作；文件的读写操作

16. spark中job，stage和task是怎么切分的?

    job：一个action算子对应一个job

    stage：从一个action算子的终点开始，从后往前看是否发生宽依赖，比如reducebykey操作。

    task：spark的最小执行单元，由分区数来决定。

    并行度：取决于executor中core的数量，可以手动设置最大并行度。

17. 对于宽窄依赖的理解

    宽窄依赖是指对于RDD中分区之间依赖，例如宽依赖是指子RDD的分区依赖多个父RDD的分区（多对一的关系）；窄依赖是一对一的关系。

18. Spark中的容错机制是怎么做到的?

    分布式数据集的容错方式有两种：数据的备份操作和记录数据的更新。spark采用的是记录数据的更新操作，因为网络的带宽往往比内存的带宽小得多。spark使用血缘关系来记录数据的生成记录，因此如果一个RDD的数据消失了，我们只需要根据父RDD进行重新的计算即可。

Spark调优

1. 数据倾斜调优
2. 小文件调优
3. 内存问题调优（driver和executor内存分配）
4. 广播机制调优
5. AQE
    1. 动态调整分区数量。
    2. 动态检测热点值，实现skew join操作。
    3. 动态实现sort merge join转换为广播join。

数仓理论

1.需求开发流程。

提出需求：业务方提出需求

需求分析：产品经理来进行需求分析

需求承接：简单需求承接和复杂需求承接（参考项目问题第7点）。

需求评审：如果是很简单的需求的话，就直接和业务那边面对面进行一个需求对接；如果需求设计的内容较多并且比较复杂的话，那就会拉伸产品经理、负责人和业务方进行一个业务需求会议。

ETL开发设计：

- 表模型的设计：设计表名、字段名；设计表的主键和外键；设计字段注释；设计表分区等。
- Mapping文档的设计：每个字段的生成逻辑；表与表之间的关系，数据链路设计。

  将上述的内容都产出ETL文件进行留存，为之后的开发流程做参考。
- 调度设计：根据具体的上下游依赖关系来设计调度任务。其中一个节点只生成一个表；多并行，少串行。

自测：

- 检查关联关系和过滤条件是否正确：join采用的是全连接还是左外连接；on的条件是否正确；where的条件是否正确；
- 检查对口径指标的计算是否正确：数据指标分为可累加和不可累加的数据口径；可累加的数据口径使用sum；不可累加的使用count。
- 检查是否能够进行优化：比如列裁剪，分区裁剪，谓词下推，避免使用count(distinct)等。

数据验收：业务方检查是否符合自己的一个需求。

上线。

2. 数据仓库的特点和好处。

   特点：面向主题的，集成的，相对稳定的，能够反映历史趋势。

   好处：

    - 集成了多个数据源，提供了统一的视图。
    - 保存历史数据，能够反应历史变化趋势，为业务决策提供了支撑。
    - 采用了数据分层，提高了数据的可复用性。

3. 说一下原子指标和派生指标的区别。

   原子指标：基于业务时间行为下的度量。即原子指标=业务过程+度量，比如申请金额。

   派生指标：派生指标=一个原子指标+修饰词+时间周期。比如一天内滴答渠道的申请总金额。

4. 数据仓库和数据库的区别。

   数据仓库的出现不是为了替代数据库，而是数据库发展到一定程度后的必然产物。

   - 设计目标：数据仓库是OLAP型，是为了分析数据；数据库是OLTP型，用于处理事务。
   - 数据处理方式：数据仓库注重ETL，不会轻易的修改数据；数据库常见的操作是增删查改。
   - 应用场景不同：数据仓库用于BI数据报表；数据库一般用于web应用开发等。
   - 用户群体不一样：数据仓库的用户群体是数据分析人员；数据库的用户群体是开发人员。

5. 两种数仓架构的区别。

   - 设计思路：Immon采用自上而下的方式，以数据作为驱动；Kimball是自下而上的，以业务需求为驱动。
   - 建模方式：Immon采用3NF建模，减少数据冗余，来提高数据的准确性和一致性；Kimball采用维度建模的方式，通过冗余数据来提高查询效率。
   - 周期：Immon需要较长的周期，使用大公司；kimball以业务需求为导向，短平快，使用小公司。
   - Kimball的灵活性更强。

6. 数仓分层的作用。

   - 屏蔽了原始数据的干扰。
   - 保证数据血缘关系可追踪，出现问题可以快速的排查并解决问题，降低了开发和维护的成本。
   - 减少烟囱式开发，每一层的职责都比较明确。
   - 提高的数据的准确性和一致性，从下往上进行开发，使用相同的下层表进行开发。
   - 增加了数据的可复用性，降低存储成本。

6. 数仓层次调用的规范。

   - 不能跨层调用。
   - ads层不能相互调用。
   - 任务的层数不能过深。

7. 两种数仓建模的区别。

   - 范式建模（减少数据冗余来提高数据的准确性和一致性，适用于OLTP）
   - 维度建模（通过增加数据冗余来提高查询效率，适用于OLAP）

拓展：

ER模型，维度数据模型和data vault模型的区别：ER的具体实例就是3NF，通过消除数据冗余追求数据的一致性；维度建模通过冗余数据减少表join次数，来提高查询的效率；data vault模型是ER的拓展，由中心表（记录主键）、链接表（连接不同的中心表）和附属表（记录一些附属的信息）组成。

8. 维度建模的四个步骤。

   以授信申请表为例

   识别业务过程：用户在需要通过授信来获取具体的额度。

   确定粒度大小：每一行记录就是用户的一次申请记录。

   确定维度：渠道维度，日期维度

确定事实：申请的金额

9. 总线矩阵的作用。

   总线矩阵的横坐标是业务过程，纵坐标是维度。

   - 为企业提供了统一的数据视图，避免重复开发。
   - 指导分层建设：可以从总线矩阵中快速找到需要的维度。
   - 支持跨主题域进行分析：不同的主题域可以通过相同维度进行分析。
   - 为新人提供了一个快速了解公司业务过程的文档。

数据域-业务过程		一致性维度					
		客户	活动	活动渠道	产品	额度类型	风控策略类型
展业域	创建活动		√				
	搭建渠道			√			
	参加活动	√	√	√	√		
授信域	授信申请	√			√	√	
	授信审批	√			√	√	
	定额/利率	√			√	√	
	提额	√				√	
风控	风险评估	√					√
交易域	借款	√			√		
	放款	√			√		
	还款	√			√		
	逾期	√			√		
催收域	催收	√					

10. 宽表的好处和局限性。

    好处：

    - 提高查询性能。
    - 提高开发效率。
    - 有效的解决口径不一致的现象。

    坏处：

    - 生产时性能较差。
    - 开发难度高。
    - 维护成本高。
    - 稳定性较差。

11. 数据模型设计的基本原则。

    - 高内聚低耦合：将相同业务过程放在同一个主题域中。
    - 核心模型和边缘模型分离：可以将核心字段放在一起，非核心字段另外进行存储。
    - 公共处理逻辑下沉：将ADS需要用的公共指标放在公共层中。
    - 口径一致性。
    - 命名清晰。

12. 如何保证指标的一致性。

    - 命名一致性：使用规定的命名规范。
    - 口径一致性：将指标的口径维护到平台上进行管理。

    表命名相关规范：层级+数据域简称+业务过程+分区类型

13. 星型模型、雪花模型和星座模型的区别。

    星型模型是一张表与多张维度表直接关联。

雪花模型是维度表之间也会关联，再与事实表进行关联。

星座模型核心的维度表会和多个事实表进行关联。

企业中我们常用的是星星模型和星座模型。

优缺点：

- 数据冗余来提高数据的一致性。
- 提高查询性能。
- 设计复杂度。

维度	星型模型	雪花模型	星座模型
数据冗余	高（非规范化维度）	低（规范化维度）	中等（共享维度可能冗余）
查询性能	高（宽表，少JOIN）	低（多表JOIN）	中等（共享维度减少部分JOIN）
结构复杂度	简单	中等	高
维护成本	低	中等	高
适用场景	单一业务过程快速分析	多层级维度分析	企业级多业务过程整合

14. 强规则和弱规则：强规则会直接终止任务；弱规则会直接进行相应的报警处理。
15. DQC的配置：

    表级别：数据量的浮动百分比。

    字段级别：是否为空，是否唯一，是否出现异常值。
16. 大概多少百分比的任务会触发DQC报警：4%以内都可以，一般SLA报警会比准确性的报警要多一点。
17. 数据域的划分遵循的原则：
    - 基于业务流程来进行拆分：比如我们公司会按照业务流程来划分客户域，申请域，授信域，风控域和贷后催收域。
    - 高内聚低耦合：将相关的业务过程的表划分到一起。
    - MECE原则（相互独立，完全穷尽）：数据域保持互斥；能够包含当前所有的业务过程；考虑之后可能加入进来的业务。
18. 维度退化一般是在哪里？

    一般维度退化会发生在dwd层和dws层。我们将常用的维度退化到事实表中：想要减少join的次数从而提高查询效率，并且也可以减少join的加工逻辑；但是维度退化也会导致一些问题，比如增大数据的存储量，维表进行更新的时候更新困难。
19. ods和dwd进行数据清洗的区别？

    ods的数据清洗是轻度清洗，一般是对明显的异常值进行处理，比如主键和外键为空的情况，或者对字符串进行标准化处理。

    dwd的数据清洗是会涉及数据验证，维度退化，以及将多张表进行关联操作等。
20. 如何评价数仓的好坏？

    技术角度（又快又好，还要便宜安全）：
    - 质量方面：是否存在过多的失败任务数；数据的准确性；数据的完整性；数据的一致性。
    - 效率方面：SLA成功率；平均查询效应时间。

- 成本方面：处理的数据量和消耗的成本比值。
- 安全方面：高密表的热度，如果高密表被频繁访问说明安全性不高。

业务角度（全能易用，客户满意）：

- 覆盖方面：是否包含了各个业务流程所需要的数据。比如如果业务方需要获取区域销售信息来制定获得的时候我们能够及时提供数据，这就说明我们的覆盖比较全面。
- 易用方面：如果我们的公共层的数据能够支撑下游多个业务系统进行使用的话，说明我们的易用性较强。
- 客户满意程度：从使用客户的满意程度来体现我们数仓建设的好坏。

21. 如何评价模型表的好坏?

    是否遵循我们数据模型建设的原则以及使用具体的指标来衡量。

    模型设计原则契合度：

    - 高内聚低耦合：业务类型相近的字段是否放在同一张表中，比如订单基础信息和订单支付信息。
    - 核心模型与边缘模型分离：核心模型是否值放入了核心业务字段，比如订单ID，下单时间，下单金额。
    - 公共逻辑下沉：统一的标准化处理等。
    - 数据是否可回滚：多次运行的结果是否相同。

    具体的指标：

    - 模型的完整度：DWS层和ADS层对ODS层的引用率不能超过15%。
    - 复用度：公共层的表是否被下层多次引用（>3次）。
    - 成本和性能平衡：冗余字段不能超过核心字段的30%，DWS的任务运行时间小于1.5h。

22. 如何设计一张宽表。

    - 冷热分离：将经常使用的数据冗余到宽表中。
    - 稳定和不稳定分离：将稳定表的字段进行冗余，不稳定的表字段留到ADS层进行冗余。这样不会影响到宽表的开发。
    - 主次分类：当一个宽表的数据越来越多会失去主题性，这个时候将不同主题的字段进行分离，更加聚焦主题性，也便于开发维护。

23. 怎么避免烟囱式开发?

    烟囱式开发分为表级别烟囱式开发和字段级别烟囱式开发。

    字段级别的可以建立指标字典，用于记录每个字段的口径和加工sql。

    表级别的可以建立统一的元数据平台来对表信息进行统计，强化对于元数据信息的管理。

    在开发规范上，要求开发之前都要去指标字典或者元数据平台去查看是否有一样的开发或者类似的开发。

24. 任务报错排查。

    整体的排查思路都是先看调度平台，检查是否上游表出现了问题；如果是具体任务报错，则查看yarn日志平台定位问题。hive和spark的区别是hive会启动多个application，而spark只会启动一个application。
    首先通过调度日志去查看报错信息，如果报错类型是很简单的可以直接进行处理，比如上游表缺失或者字段缺失等。如果通过调度日志无法解决则通过yarn日志进行信息排查。

hive报错任务排查：直接根据调度平台上的application id来查看是哪一个stage发生了问题，然后通过explain sql来定位stage对应哪一段sql。
在yarn日志中根据application ID来定位，查看是哪个stage发生了报错，然后通过explain查看执行计划来判断定位stage对应的sql，再使用sql调优和相关参数调优进行调整。

spark报错任务排查：在application id中找到流程图，spark直接定位sql问题需要先定位failed stage，然后再定位在dataframe中的节点，在通过节点中的显示来定位到具体的sql中。

25. 事实表的三种类型。

    事务事实表；周期快照事实表；累计快照事实表

	事务事实表	周期快照事实表	累积快照事实表
时间/时期	时间	时期	时期（较短）
粒度	基本事件	时间周期	业务周期
加载方式	新增	新增	新增+修改
时间维	操作日期	期末时点	多个操作完成日期
例子	订单支付流水	每日账户余额/常见的按日的汇总表	物流包裹状态跟踪

26. 字段应该放在哪一层?

    根据字段的复用性来确定应该放在哪一层，如果ads层多张表都复用了这个字段，那么我们就会将其设计在dws层；如果dws层的多张表都用到了这个字段那么我们就将其设计在dwd层。

27. 数据治理的角度

    - 主数据管理：数据存储管理；数据采集管理。
    - 元数据管理：元数据维护，元数据权限管理；元数据分析；血缘管理。
    - 数据质量管理：模型管理；质量规则管理。
    - 数据安全管理：权限管控；数据脱敏；安全传输。

28. 企业数据治理的流程

    1.我有什么？确定我有哪些任务，哪些DQC规则，SLA承诺等。

    2.明确治理目标：比如运行时间前100个任务；占存储大小的前100个任务。

    3.怎么治理：针对运行时间长的可以考虑优化手段；存储大小的可以考虑修改压缩格式等。

    4.衡量治理效果：做一些量化手段，比如存储压缩比是多少。

    5.总结和复盘：对整个治理的经验总结，问题归纳等。

29. 如何从0到1搭建数仓

    1.业务调研

    2.数据调研

    3.划分数据域

    4.构建业务总线矩阵

    5.确定数仓分层架构

    6.制定数仓规范

    7.梳理指标体系

    8.模型开发

    9.数仓运维保障

10.数据质量监控工作

30. 主题域和数据域的区别是什么?

	数据域	主题域
划分角度	是自下而上的，以业务数据视角来划分数据，	是自上而下的，以业务分析视角来划分数据。
应用场景	将数据按照业务视角进行划分便于管理和分析。通常存在dwd和dws层。	适用于数据集市的设计，通常存在于dws层和ads层。

31. 数据仓库和数据集市的区别?

    数据集市相当于是数据仓库的一个子集，针对与一个具体的业务线或者部门。

32. 三范式之间的区别?

    1. 第一范式原子不可分
    2. 第二范式非主键不能部分依赖主键
    3. 第三范式不能存在间接依赖。

33. 维度退化和维度下沉的区别?

    维度退化不保留维度表，维度下沉我们会保留维度表。

34. 维度表的两种类型?

    扁平维度表：维度信息也会进行冗余，一般用于星型模型。

    雪花维度表：维度表直接互相关联，一般用于雪花模型中。

35. 维度设计时维度整合和维度拆分的场景。

    维度整合：在不同业务数据库中的字段名和编码格式不同。例如A系统的男0，女1；B系统的男M，女F。
    维度拆分：

    1. 水平拆分：当一个维度表中设计的主体过多时需要进行拆分。
    2. 垂直拆分：当维度表中有些热度高，有些热度低的时候进行拆分。

36. 数据质量遇到过哪些问题?

    1. 数据缺失：核心业务字段为空或者无效内容。比如授信审批表中部分没有审批结果。
    2. 数据不一致：多个业务系统中关于相关实体的内容不一致。比如住址信息。
    3. 数据重复。
    4. 枚举值不一致。

具体项目问题：

信贷整体业务流程：

展业：营销获客、引导账号注册和信息采集

授信：授信定额（动态调整-调整/冻结额度）、利率定价

贷前风控-额度-借款-贷中风控-放款-还款-催收简单回答一下授信业务过程

1. 信贷业务主要分为三个核心环节：贷前、贷中以及贷后。授信就属于贷前阶段。
    - 首先用户注册,实名认证，绑定银行卡，然后提交授信申请，签署征信授权。

- 接着会有两轮校验，前置校验和反欺诈校验。前置校验会校验地区，年龄、反洗钱等基本信息。反欺诈校验是调用我们这边的反欺诈系统，判断恶意欺诈风险。
- 然后是信用审批，根据人行征信和外部大数据，A卡评分模型，确认是否审批通过。
- 最后根据申请的产品生成响应的额度。

2. 在实习过程中遇到的最大问题是什么？

   业务方面：

   刚开始入职的时候对公司的整体业务流程不是很了解，有点无从下手的感觉。

   首先就是梳理公司业务线的一个整体业务流程，比如在金融信贷中一般都会分成贷前-贷中-贷后三个阶段。

   - 贷前：展业和授信。
   - 贷中：额度，借款。
   - 贷后：放款，还款，催收，

   先了解整个业务流程，再细化到具体的域，这个时候我会主要通过公司沉淀的总线矩阵来帮助我进行梳理业务脉络。比如我之前在公司负责的是授信域，所以我会通过总线来看我负责的授信域主要有哪些业务流程，比如说授信申请，授信审批，利率定价等。

   然后我会通过数据探查平台来对这些业务流程的表进行一个探查，看里面主要的字段，结合指标平台来统一字段的口径。

   最后就是通过一些小的需求来进行上手开发，来进一步的巩固加深对业务流程的理解。

   技术方面：

   之前核心业务中的渠道风险报表出现了SLA报警处理，里面主要有渠道，借款金额，利率，30日违期总金额，不良率等字段。
   在处理的时候主要是看yarn日志来追踪任务，发现在ruduce端出现一个任务执行时长远大于其他的任务，并且reduce的完成度一直卡在99%，这个时候就可以判断是出现了数据倾斜。
   在处理的时候通过对渠道进行一个数据探查，在进行数据探查的时候发现unknown这个渠道字段的数量远大于其他有含义的渠道
   首先第一个想到的是和业务那边进行确定看可不可以对渠道这个字段进行进一步挖掘，来对渠道进一步的划分，在得知不能之后；我使用负载均衡的思想，想着让更多的机器参加运算，我使用sql对热点值进行加盐处理然后和非热点值进行unoin all，之后再恢复原数据。
   进行完优化之后报表的执行时间从2小时优化到1个半小时以内，成功解决了SLA产出问题。
   经历这个事情之后我总结出了一套任务优化理论：查看yarn日志和解析执行计划来定位问题->数据探查发现热点值->和业务方尝试解决方案→sql代码优化。

3. 在实习过程中你最大的收获是什么？

   在实习过程中总结出了一套自己的'性能问题排查与解决'的方法论。

   首先通过查看yarn任务日志和解析执行计划来定位问题。

   然后来解决具体的问题，比如数据倾斜问题的话会首先和业务区进行沟通，看能不能对里面的一些字段进行细化。如果不能的话就会通过一些常见的sql优化和参数优化来进行调优。

4. 面试之后的反问？

   - 技术栈？
   - 主要的业务流程，如果我进去之后负责哪一块的工作？
   - 公司是否有关于agent结合数据开发的应用探索。

5. 开发环境：Hive+Spark+小海豚+presto+FineBI

   服务器配置：64core+256内存+12*8T硬盘。

数据存储规模：2-3PB，3副本机制。

任务个数：1000多个脚本，每天会有几千个app任务调度执行。

表分布个数：ODS 2000+， DWD 1600+， DWS 600+， ads 1000+，DIM 250+。

一个模糊的概念：2千万行数据，160个字段的客户信息表，每个日分区的大小3.5GB。

6. 数据倾斜问题有遇到过吗？是怎么解决的?

   遇到过，数据倾斜分为map端和reduce端的数据倾斜。

   map端是因为文件分布不均且不可拆分。

   reduce端分为group by和join两种类型。

   在解决之前都可以查看是否可以进行提前的数据过滤或者聚合来减少数据量。然后再进行排查。

7. 需求承接/需求评审/需求排期

   我们公司进行需求承接一般都是在jira上发布需求，其中会包括需求背景，需求内容，产出物是什么等。

   如果需求比较简单的话就会直接和产品经理进行需求确定，然后就开始开发；如果需求比较复杂的话我们会开一个专门的需求评审会来确定需求，其中包括业务方，产品经理，数仓负责人，数据分析师和开发人员，主要确定需求的可行性，指标的口径，需求排期等内容。

8. 数据质量管理

   事前监控：

   - 数据源评估。
   - 数据清洗：。

   事中监控（DQC规则）：

   - DQC规则配置

   事后监控：

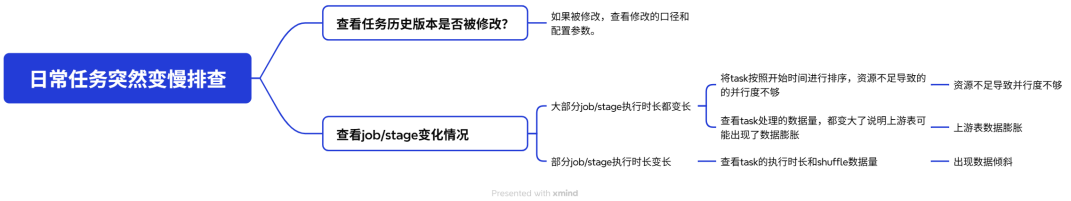
   - 每隔一段时间对数据质量问题进行排查，然后进行补救等工作。

   拓展：

   1. DQC规则配置的参考依据（业务角度，企业数据标准，具体开发实现）
      1. 业务规则：根据具体业务来指定标准，比如申请金额不能是负数，申请状态不能为空等。
      2. 企业数据标准：企业数据字典的标准，比如时间格式是否满足公司标准。
      3. 技术实现规范：结合具体的开发口径来配置相关规则。比如授信通过率不能大于100%。

9. 日常任务突然变慢怎么排查

   1. 首先要对代码历史发布日志进行检查，检查是否有人修改了代码。如果发生修改，查看修改的内容是否会影响任务的执行。
   2. 查看任务运行日志和之前日志的区别：
      - 如果出现大部分job都出现了任务延时的情况，可以从资源不足导致并行度不够（查看task之间的启动时间是否差的很多）和数据膨胀导致task处理数据量过大（查看task的数据输入量）。
      - 如果出现少部分job出现任务延时的情况，可以从数据倾斜的角度来对task任务进行排查，查看部分task的数据输入量是不是显著大于其他的task。



10. 你们公司是划分了哪些数据域?

    根据业务过程我们划分了展业域，授信域，交易域和催收域。

11. 划分数据域的目的是什么?

    1. 支撑业务逻辑的映射和复用：一般数据域和金融信贷的核心业务流程一一对应的，能够准确的反映业务逻辑。这种映射关系可以让我们快速找到对应的数据位置，同时避免重复开发。
    2. 保证数据的一致性和规范性：同一数据域中的内容使用相同的字段定义，减少数据的冗余和歧义。
    3. 赋能数据分析与决策支撑：数据域的划分与业务主体紧密关联，数据分析师和业务人员能够快速理解数据含义。

12. 你们授信域的北极星指标是啥?

    授信支用率，也就是实际使用的额度 / 授信额度。

    拓展：

    授信集中度的含义是什么：用户或者机构的授信额度占资本净额的比例，不能过高防止风险过大。

13. 一个慢大任务怎么进行优化

    首先分析任务变慢的类型是什么：是一直变慢还是在间歇性变慢；变慢时是部分job变慢还是整体进行变慢。

    分析原因和具体解决方案：

    - 上游依赖优化：是否由于上游表产出不及时导致任务一直处于pending状态。
    - 资源不足导致：提高优先级，提高队列权重
    - 任务层面拆分：将一个复杂的任务拆分成如干个可以并行的任务
    - 代码层面优化：sql优化之类的
    - 参数配置优化：增大资源
    - 数据倾斜问题：

14. 在数仓建设中遇到过什么难题和问题?

    数据源混乱，职责不清，业务系统各自为政，形成了严重的数据壁垒。

    以客户授信额度举例：

    S：在核心系统中，会有“合同额度”字段，这代表着理论上的授信上限。比如客户a的合同额度是100w。

    风控系统中会有“动态额度”字段，这代表在评估客户风险之后，实际的放款校验。比如客户a经过评估之后只有60w。

    在我们计算授信支用率的时候，用于这两个字段本身代表的含义不一样，所以有时候统计出来的支用率会有比较大的偏差。然后导致业务方那边进行分析的时候对我们的数据产生了质疑，但是这个实际上是由于数据源导致的问题。

    T：由于这两个字段在之后的分析中都是有作用的，所以我们的任务不是将其强行归类，而是都进行保留并且理清楚这两个字段分别参与指标计算时分别代表什么样的含义，方便在之后给业务进行使用的时候能够按照他们的要求来提供数据。

A：我们联合授信团队和风控团队一起明确了这两个指标的开发口径（法定支用率和风控支用率）和含义。将这两个字段都放在同一表中，并且明确各个字段在哪些指标开发中会使用到，记录在数据字典和指标体系中。

R：成功解决了由于业务数据库导致的数据含义模糊的情况，并且能够根据具体场景来选择对应的指标开发，并且之后再未出现过因口径不清而导致的投诉。

拓展：

法定支用率和风控支用率的作用：

法定支用率：一般用于监管部门和业务管理部门，我们不能超过法定的合同额度。

风控支用率：为实时放款决策做分析；以及风控管理。

15. 数仓链路优化的经验
    1. 提高链路的并行度：通过分析节点依赖关系，尽量让任务多并行，少串行。
    2. 优化资源分配和任务调度：优先生成重要的任务；或者调整调度时间，避免和关键任务争夺资源。
    3. 通过sql优化来缩短单个节点的执行时间。
16. 数仓建设会有什么问题？

    数据质量问题：在dws层还会出现一些数据质量的问题，比如日期编码格式问题，申请金额出现异常值。需要在dwd层就确保数据的质量。

    命名不统一：相同开发口径的字段有多个名字。需要有一个专门的术语表，然后指定严格的命名规范来解决。

    数据一致性问题：不同表中相同字段的开发口径不一致。比如用户授信总额在授信域中按照合同额度求和，在客户域中按照动态额度求和。需要使用指标字典来统一指标的开发口径。